

Práctica 0 — MATI

Matemática Aplicada a las Tecnologías de la Información · Grado en Ingeniería
Informática · Universidad de Sevilla · Curso 2026/2027

Conjuntos de enteros con restricciones sobre las diferencias (inspirado en un pasatiempo de *Le Monde* 2, nº 6).

⚙️ **Kernel:** este notebook usa el kernel `Python (MATI .venv)`. Si al ejecutar aparece `ModuleNotFoundError`, ve a **Kernel** → **Change Kernel** y selecciónalo (ver `GUIA_USO.md` §1.2).

```
In [1]: # Librerías que se usan en todo el notebook + funciones de dibujo compartidas
import networkx as nx          # crear y analizar grafos
import matplotlib.pyplot as plt # dibujar (uso directo solo si mati_utils no basta)
import sympy as sp             # cálculos matemáticos exactos
import numpy as np              # operaciones numéricas auxiliares

from mati_utils import dibujar_grafo, dibujar_multigrafo, SEED # funciones de dibujo
```

Ejercicio 1 — Conjuntos y diferencias que son cuadrados perfectos

📄 ENUNCIADO

El número x se define como las dos últimas cifras de tu DNI sumadas a 50. (Para este enunciado, tomando como referencia el DNI con terminación 29, se tiene $x = 79$).

- a) ¿Cuál es el tamaño del mayor conjunto $S \subseteq [0, \dots, x]$ de forma que no contiene dos enteros i, j tal que $|i - j|$ es un cuadrado perfecto?
- b) ¿Cuál es el tamaño del mayor conjunto $T \subseteq [0, \dots, x]$ de forma que dados dos enteros i, j de T , $|i - j|$ es un cuadrado perfecto? ¿Por qué es siempre ese número?

(Inspirado en un pasatiempo del número 6 de “Le Monde 2”)

Requisitos de entrega del departamento: intento de resolución sin IA · uso de IA explicado (prompt, modelo utilizado) · discusión sobre si la respuesta de la IA es correcta y por qué · en su caso, respuesta correcta/corregida · ampliaciones del problema usando IA.

TEORÍA

Cuadrado perfecto. Un entero que es el cuadrado de otro entero: $1, 4, 9, 16, 25, \dots$ (también $0 = 0^2$, pero como aquí las diferencias son entre números *distintos*, nunca salen 0).

La idea clave: es un problema de grafos disfrazado. Construimos un grafo G cuyos *vértices* (los “puntos”) son los números $0, 1, \dots, x$, y ponemos una *arista* (una “línea”) entre i y j exactamente cuando $|i - j|$ es un cuadrado perfecto. Dicho corto: **dos números están unidos si su diferencia es un cuadrado**.

Con esa traducción, los dos apartados piden dos cosas clásicas de teoría de grafos:

- **Conjunto independiente** = grupo de vértices entre los que *no* hay ninguna arista. El apartado **a)** pide el mayor de G ; su tamaño se llama *número de independencia*, $\alpha(G)$.
- **Clique** (o subgrafo completo) = grupo de vértices en el que *todos* están unidos con todos. El apartado **b)** pide la mayor de G ; su tamaño se llama *número de clique*, $\omega(G)$.

Ejemplo pequeño con $x = 5$ (vértices $0..5$; cuadrados que caben: 1 y 4):

- Aristas por diferencia 1: $0-1, 1-2, 2-3, 3-4, 4-5$. Aristas por diferencia 4: $0-4, 1-5$.
- $\{0, 2\}$ sí es independiente, porque $|0 - 2| = 2$ no es cuadrado. $\{0, 1\}$ no lo es, porque $|0 - 1| = 1 = 1^2$.
- $\{0, 1, 5\}$ no es clique: $0-1$ y $1-5$ son aristas, pero $|0 - 5| = 5$ no es cuadrado.

Dos avisos prácticos. (1) Calcular $\alpha(G)$ en un grafo cualquiera es NP-duro, así que en a) lo razonable no es probar conjuntos al azar sino buscar un *patrón periódico* de residuos que nunca produzca una diferencia cuadrada, y luego comprobar computacionalmente que es óptimo. (2) En b) la condición es mucho más fuerte (*todas* las diferencias entre pares deben ser cuadrados): conviene ver primero cuántos números pueden convivir a la vez y por qué no caben más — ahí está la respuesta al “¿por qué es siempre ese número?”.

DATOS

Del enunciado solo hay que fijar el valor de x , que depende del DNI:

- **Dos últimas cifras del DNI:** `DNI_DOS_ULTIMAS` = 64.
- $x = \text{DNI_DOS_ULTIMAS} + 50 \rightarrow x = 64 + 50 = 114$.
- **Universo de trabajo:** los enteros $[0, 1, \dots, x]$, o sea $x + 1$ números en total (115 en nuestro caso). Ojo: son $x + 1$, no x — el 0 cuenta.

- **Cuadrados perfectos relevantes:** los $k^2 \leq x$ con $k \geq 1$, porque una diferencia dentro del universo nunca puede pasar de x .

```
In [2]: # Dato de partida del enunciado: Las dos últimas cifras del DNI.
DNI_DOS_ULTIMAS = 64

# x = dos últimas cifras del DNI + 50; el universo son los enteros de 0 a x, ambos
x = DNI_DOS_ULTIMAS + 50
universo = list(range(0, x + 1))

# Cuadrados perfectos que pueden aparecer como diferencia |i - j| dentro del univer
cuadrados = [k**2 for k in range(1, int(x**0.5) + 1)]

# Resumen de los datos de partida
print(f"x = {DNI_DOS_ULTIMAS} + 50 = {x}")
print(f"universo [0..x]          : {len(universo)} enteros -> {universo[:5]} ...")
print(f"cuadrados perfectos <= x : {cuadrados}")
print(f"mayor cuadrado alcanzable : {cuadrados[-1]} = {int(x**0.5)}^2")
```

```
x = 64 + 50 = 114
universo [0..x]          : 115 enteros -> [0, 1, 2, 3, 4] ... [112, 113, 114]
cuadrados perfectos <= x : [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
mayor cuadrado alcanzable : 100 = 10^2
```

RESOLUCIÓN

Trabajo del estudiante

a) *Tamaño del mayor conjunto $S \subseteq V$ sin diferencias que sean cuadrados perfectos.*

Definiciones de partida.

- Un número $n \in \mathbb{Z}$ es un **cuadrado perfecto** si $\exists k \in \mathbb{Z}$ tal que $k^2 = n$.
- $V = \{0, 1, \dots, x\}$, donde x son las dos últimas cifras del DNI del alumno + 50 (en este caso $64 + 50 = 114$).
- $A = \{(i, j) : i, j \in V, i \neq j, |i - j| \text{ es un cuadrado perfecto}\}$.

El grafo $G = (V, A)$ representa qué cuadrados perfectos pueden construirse como diferencia de dos elementos distintos de V : **una arista entre dos vértices significa que la diferencia de esos dos elementos es un cuadrado perfecto.**

Además, el mayor cuadrado perfecto que puede construirse como diferencia de dos elementos de V es

$$\lfloor \sqrt{x} \rfloor^2.$$

Procedimiento para construir $S \subseteq V$ de forma que no contenga dos enteros i, j con $|i - j|$ cuadrado perfecto:

1. Tratamos V como una **lista ordenada**.
2. Hallamos el listado de todos los cuadrados perfectos que pueden construirse como $|i - j|$ con $i, j \in V$.
3. Comenzamos por el primer elemento V_0 y eliminamos todos los elementos V_i tales que $|V_i - V_0|$ es un cuadrado perfecto (el propio V_0 se conserva).
4. Al subconjunto resultante le aplicamos lo mismo con V_1 : eliminamos todo V_i tal que $|V_i - V_1|$ es un cuadrado perfecto.
5. Ejecutamos esto iterativamente hasta que el índice del elemento a analizar sea el máximo posible.
6. El subconjunto resultante de todas las iteraciones es el mayor conjunto independiente contenido en el grafo, y su tamaño es el **número de independencia** $\alpha(G)$, que es lo que pide el apartado a).

b) *Tamaño del mayor conjunto $T \subseteq V$ en el que toda diferencia entre dos elementos es un cuadrado perfecto.*

Lo que se busca es el **clique de mayor tamaño** del grafo G ; en concreto su tamaño, que es el **número de clique** $\omega(G)$.

Una forma rápida de obtenerlo por código es aprovechar el método que facilita la biblioteca `networkx`: `nx.find_cliques`, iterando sobre los resultados hasta encontrar los de mayor tamaño.

Matemáticamente podemos razonar que el tamaño **siempre va a ser 3**, partiendo de que para cualquier alumno el valor de las dos cifras de su DNI es como mucho 99, luego el valor máximo de x es 149.

1. Clique de tamaño 1. Es trivial: cualquier vértice por sí mismo es un grafo completo K_1 . Además no contaría para el resultado, porque no podría calcularse la diferencia con otro nodo, lo que imposibilita que esa distancia sea un cuadrado perfecto.

2. Clique de tamaño 2. Sigue siendo trivial: basta escoger dos nodos cualesquiera que sean adyacentes entre ellos. Por ejemplo $(0, 1)$, $(0, 4)$, $(1, 3)$, $(1, 10)$, $(2, 6)$, ...

3. Clique de tamaño 3 (el óptimo y máximo para este valor de x). Aquí el asunto se vuelve más complejo. Buscamos tres nodos a, b, c que cumplan simultáneamente que $|a - b|$, $|b - c|$ y $|a - c|$ son cuadrados perfectos. Llamemos a esas distancias:

$$|a - b| = x^2, \quad |b - c| = y^2, \quad |a - c| = z^2.$$

Además tenemos que $|a - b| + |b - c| = |a - c|$, que es lo mismo que

$$x^2 + y^2 = z^2,$$

esto es, una **terna pitagórica**. Por lo tanto, para la solución de 3 nodos basta con encontrar una terna pitagórica dentro del conjunto. Existen múltiples soluciones: desde

$(3, 4, 5)$ hasta cualquiera que no supere el rango de 149. Es decir, la primera solución sería:

$$|a - b| = x^2 = 3^2 = 9, \quad |b - c| = y^2 = 4^2 = 16, \quad |a - c| = z^2 = 5^2 = 25, \\ a = 0, \quad b = 9, \quad c = 25.$$

4. Clique de tamaño 4. Se complica bastante y se hace imposible en el rango marcado. Buscamos cuatro nodos a, b, c, d que cumplan simultáneamente que $|a - b|$, $|b - c|$, $|a - c|$ y $|c - d|$ son cuadrados perfectos. Llamemos a esas distancias:

$$|a - b| = x^2, \quad |b - c| = y^2, \quad |a - c| = z^2, \quad |c - d| = t^2, \quad |a - d| = u^2.$$

Además tenemos que

$$|a - b| + |b - c| = |a - c| \implies x^2 + y^2 = z^2,$$

$$|a - c| + |c - d| = |a - d| \implies z^2 + t^2 = u^2.$$

Eso implica que **la hipotenusa de la primera terna pitagórica debe ser cateto de la segunda**. Partiendo del caso más pequeño ($a = 0, b = 9, c = 25$, con $x = 3, y = 4, z = 5$), eso implica que

$$25 + t^2 = u^2, \quad \text{siendo } 5 < t < u.$$

La única terna pitagórica que comienza en 5 es la $(5, 12, 13)$, la segunda más pequeña de todas. Comprobamos que para esa solución (la menor posible):

$$|a - b| = 3^2 = 9, \quad |b - c| = 4^2 = 16, \quad |a - c| = 5^2 = 25, \quad |c - d| = 12^2 = 144, \\ |a - d| = 13^2 = 169.$$

Esto hace imposible esta solución, ya que en el rango marcado no hay posibilidad de que existan dos nodos a y d cuya distancia sea mayor que 149, por lo que 169 quedaría fuera del rango establecido.

Se consolida así que el número de clique para cualquier grafo de las características propuestas es

$$\omega(G) = 3.$$

CÓDIGO

Trabajo del estudiante

```
In [3]: # =====
# APARTADO a) - mayor conjunto independiente S del grafo G
# =====

# --- 1. Construcción del grafo G = (V, A) ---
# Vértices: el universo [0..x]. Aristas: pares (i, j) cuya diferencia es un cua
```

```

G = nx.Graph()
G.add_nodes_from(universo)

aristas = [(i, j) for i in universo for j in universo if i < j and abs(i - j) in cu
G.add_edges_from(aristas)

print(f"Grafo G: {G.number_of_nodes()} vértices, {G.number_of_edges()} aristas")
print("-" * 60)

# --- 2. Construcción iterativa de S ---
# Partimos de S = V y, tomando cada elemento como referencia, eliminamos de S
# todos los que estén a distancia igual a un cuadrado perfecto del de referenci
S = universo.copy()

for k in range(len(S)):
    elementos_a_eliminar = [i for i in S if abs(i - S[k]) in cuadrados]
    for i in elementos_a_eliminar:
        S.remove(i)

    # Traza de la iteración: elemento de referencia y subconjunto que queda tras de
    print(f"iteración {k + 1:>2} | referencia = {S[k]:>3} | |S| = {len(S):>3} | S =

    # Nos detenemos cuando el elemento de referencia es ya el último de la lista
    if S.index(S[k]) >= len(S) - 1:
        break

print("-" * 60)
print(f"Conjunto independiente S : {S}")
print(f"Tamaño |S| (voraz) : {len(S)}")

# --- 3. Visualización del subgrafo inducido por S ---
# Al ser S independiente, el dibujo debe salir sin ninguna arista.
G_S = G.subgraph(S)
dibujar_grafo(G_S, titulo=f"Subgrafo inducido por S (|S| = {len(S)}, x = {x})");

```

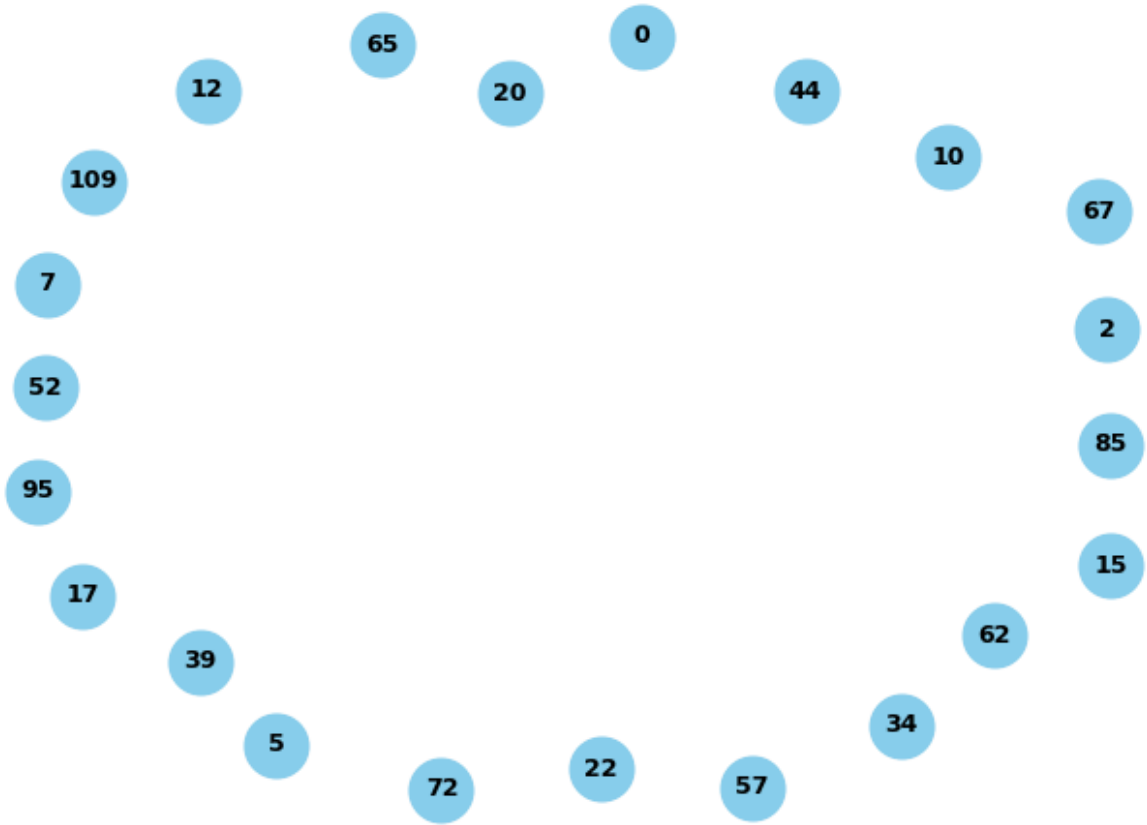
Grafo G: 115 vértices, 765 aristas

```
-----
iteración 1 | referencia = 0 | |S| = 105 | S = [0, 2, 3, 5, 6, 7, 8, 10, 11, 12,
13, 14, 15, 17, 18, 19, 20, 21, 22, 23, 24, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35,
37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 50, 51, 52, 53, 54, 55, 56, 57, 58,
59, 60, 61, 62, 63, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80,
82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 101, 102, 10
3, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114]
iteración 2 | referencia = 2 | |S| = 95 | S = [0, 2, 5, 7, 8, 10, 12, 13, 14, 1
5, 17, 19, 20, 21, 22, 23, 24, 26, 28, 29, 30, 31, 32, 33, 34, 35, 37, 39, 40, 41, 4
2, 43, 44, 45, 46, 47, 48, 50, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 65, 6
7, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 82, 84, 85, 86, 87, 88, 89, 9
0, 91, 92, 93, 94, 95, 96, 97, 98, 99, 101, 103, 104, 105, 106, 107, 108, 109, 110,
111, 112, 113, 114]
iteración 3 | referencia = 5 | |S| = 87 | S = [0, 2, 5, 7, 8, 10, 12, 13, 15, 1
7, 19, 20, 22, 23, 24, 26, 28, 29, 31, 32, 33, 34, 35, 37, 39, 40, 42, 43, 44, 45, 4
6, 47, 48, 50, 52, 53, 55, 56, 57, 58, 59, 60, 61, 62, 63, 65, 67, 68, 70, 71, 72, 7
3, 74, 75, 76, 77, 78, 79, 80, 82, 84, 85, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 9
7, 98, 99, 101, 103, 104, 106, 107, 108, 109, 110, 111, 112, 113, 114]
iteración 4 | referencia = 7 | |S| = 79 | S = [0, 2, 5, 7, 10, 12, 13, 15, 17, 1
9, 20, 22, 24, 26, 28, 29, 31, 33, 34, 35, 37, 39, 40, 42, 44, 45, 46, 47, 48, 50, 5
2, 53, 55, 57, 58, 59, 60, 61, 62, 63, 65, 67, 68, 70, 72, 73, 74, 75, 76, 77, 78, 7
9, 80, 82, 84, 85, 87, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 101, 103, 104, 10
6, 108, 109, 110, 111, 112, 113, 114]
iteración 5 | referencia = 10 | |S| = 71 | S = [0, 2, 5, 7, 10, 12, 13, 15, 17, 2
0, 22, 24, 28, 29, 31, 33, 34, 37, 39, 40, 42, 44, 45, 47, 48, 50, 52, 53, 55, 57, 5
8, 60, 61, 62, 63, 65, 67, 68, 70, 72, 73, 75, 76, 77, 78, 79, 80, 82, 84, 85, 87, 8
9, 90, 92, 93, 94, 95, 96, 97, 98, 99, 101, 103, 104, 106, 108, 109, 111, 112, 113,
114]
iteración 6 | referencia = 12 | |S| = 63 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 24, 29, 31, 33, 34, 39, 40, 42, 44, 45, 47, 50, 52, 53, 55, 57, 58, 60, 62, 63, 6
5, 67, 68, 70, 72, 73, 75, 77, 78, 79, 80, 82, 84, 85, 87, 89, 90, 92, 94, 95, 96, 9
7, 98, 99, 101, 103, 104, 106, 108, 109, 111, 113, 114]
iteración 7 | referencia = 15 | |S| = 58 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 29, 33, 34, 39, 42, 44, 45, 47, 50, 52, 53, 55, 57, 58, 60, 62, 63, 65, 67, 68, 7
0, 72, 73, 75, 77, 78, 80, 82, 84, 85, 87, 89, 90, 92, 94, 95, 97, 98, 99, 101, 103,
104, 106, 108, 109, 111, 113, 114]
iteración 8 | referencia = 17 | |S| = 54 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 29, 34, 39, 44, 45, 47, 50, 52, 55, 57, 58, 60, 62, 63, 65, 67, 68, 70, 72, 73, 7
5, 77, 78, 80, 82, 84, 85, 87, 89, 90, 92, 94, 95, 97, 99, 101, 103, 104, 106, 108,
109, 111, 113, 114]
iteración 9 | referencia = 20 | |S| = 50 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 34, 39, 44, 47, 50, 52, 55, 57, 58, 60, 62, 63, 65, 67, 68, 70, 72, 73, 75, 77, 7
8, 80, 82, 85, 87, 89, 90, 92, 94, 95, 97, 99, 103, 104, 106, 108, 109, 111, 113, 11
4]
iteración 10 | referencia = 22 | |S| = 47 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 34, 39, 44, 50, 52, 55, 57, 60, 62, 63, 65, 67, 68, 70, 72, 73, 75, 77, 78, 80, 8
2, 85, 87, 89, 90, 92, 94, 95, 97, 99, 104, 106, 108, 109, 111, 113, 114]
iteración 11 | referencia = 34 | |S| = 45 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 34, 39, 44, 52, 55, 57, 60, 62, 63, 65, 67, 68, 72, 73, 75, 77, 78, 80, 82, 85, 8
7, 89, 90, 92, 94, 95, 97, 99, 104, 106, 108, 109, 111, 113, 114]
iteración 12 | referencia = 39 | |S| = 43 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 34, 39, 44, 52, 57, 60, 62, 63, 65, 67, 68, 72, 73, 77, 78, 80, 82, 85, 87, 89, 9
0, 92, 94, 95, 97, 99, 104, 106, 108, 109, 111, 113, 114]
iteración 13 | referencia = 44 | |S| = 40 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 34, 39, 44, 52, 57, 62, 63, 65, 67, 68, 72, 73, 77, 78, 82, 85, 87, 89, 90, 92, 9
```

```
4, 95, 97, 99, 104, 106, 109, 111, 113, 114]
iteración 14 | referencia = 52 | |S| = 38 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 34, 39, 44, 52, 57, 62, 63, 65, 67, 72, 73, 78, 82, 85, 87, 89, 90, 92, 94, 95, 9
7, 99, 104, 106, 109, 111, 113, 114]
iteración 15 | referencia = 57 | |S| = 35 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 34, 39, 44, 52, 57, 62, 63, 65, 67, 72, 78, 85, 87, 89, 90, 92, 94, 95, 97, 99, 1
04, 109, 111, 113, 114]
iteración 16 | referencia = 62 | |S| = 31 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 34, 39, 44, 52, 57, 62, 65, 67, 72, 85, 89, 90, 92, 94, 95, 97, 99, 104, 109, 11
3, 114]
iteración 17 | referencia = 65 | |S| = 29 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 34, 39, 44, 52, 57, 62, 65, 67, 72, 85, 89, 92, 94, 95, 97, 99, 104, 109, 113]
iteración 18 | referencia = 67 | |S| = 28 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 34, 39, 44, 52, 57, 62, 65, 67, 72, 85, 89, 94, 95, 97, 99, 104, 109, 113]
iteración 19 | referencia = 72 | |S| = 27 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 34, 39, 44, 52, 57, 62, 65, 67, 72, 85, 89, 94, 95, 99, 104, 109, 113]
iteración 20 | referencia = 85 | |S| = 25 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 34, 39, 44, 52, 57, 62, 65, 67, 72, 85, 95, 99, 104, 109, 113]
iteración 21 | referencia = 95 | |S| = 23 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 34, 39, 44, 52, 57, 62, 65, 67, 72, 85, 95, 109, 113]
iteración 22 | referencia = 109 | |S| = 22 | S = [0, 2, 5, 7, 10, 12, 15, 17, 20, 2
2, 34, 39, 44, 52, 57, 62, 65, 67, 72, 85, 95, 109]
```

Conjunto independiente S : [0, 2, 5, 7, 10, 12, 15, 17, 20, 22, 34, 39, 44, 52, 57, 62, 65, 67, 72, 85, 95, 109]
Tamaño |S| (voraz) : 22

Subgrafo inducido por S (|S| = 22, x = 114)



```
In [4]: # =====
# APARTADO b) – mayor clique T del grafo G (número de clique)
```

```

# =====
import itertools # combinaciones de pares, para recorrer T sin repetir parejas

# --- 1. Cliques maximales de G, vía networkx ---
# nx.find_cliques devuelve todos los cliques maximales; nos quedamos con los de
cliques = list(nx.find_cliques(G))
tam_max = max(len(c) for c in cliques)
maximos = [sorted(c) for c in cliques if len(c) == tam_max]

print(f"Cliques maximales encontrados : {len(cliques)}")
print(f"Tamaño del mayor T =  $\omega(G)$  : {tam_max}")
print(f"Nº de conjuntos T óptimos : {len(maximos)}")
print(f"Ejemplos : {maximos[:5]}")
print("-" * 60)

# --- 2. Verificación explícita de que el T hallado cumple el enunciado ---
# Todas las diferencias entre pares de T deben ser cuadrados perfectos.
T = maximos[0]
for i, j in itertools.combinations(T, 2):
    print(f" |{j} - {i}| = {abs(j - i):>3} = {int(abs(j - i) ** 0.5)}²")

print("-" * 60)
print(f"Conjunto T : {T}")
print(f"Tamaño |T| =  $\omega(G)$  : {len(T)}")

# --- 3. Visualización del clique máximo ---
# Al ser T un clique, el dibujo sale como un triángulo: todos unidos con todos.
dibujar_grafo(G.subgraph(T), titulo=f"Subgrafo inducido por T (|T| = {len(T)}), x =

```

Cliques maximales encontrados : 605

Tamaño del mayor T = $\omega(G)$: 3

Nº de conjuntos T óptimos : 210

Ejemplos : [[1, 65, 101], [1, 37, 101], [1, 10, 26], [1, 17, 26], [2, 66, 102]]

```

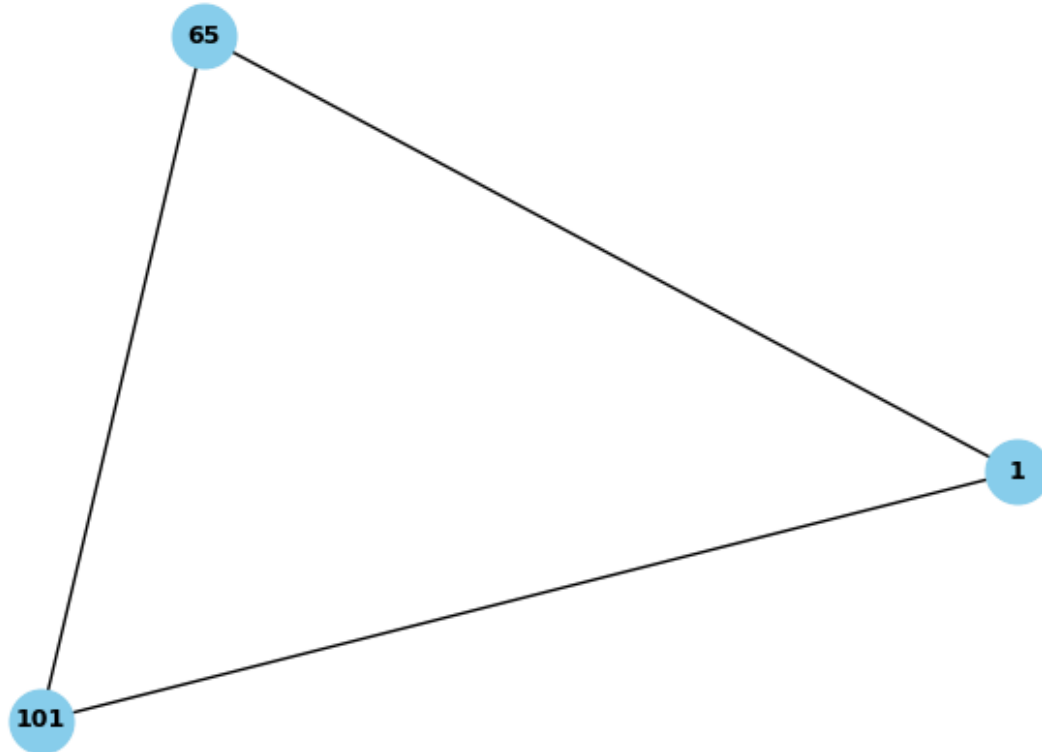
-----
|65 - 1| = 64 = 8²
|101 - 1| = 100 = 10²
|101 - 65| = 36 = 6²
-----

```

Conjunto T : [1, 65, 101]

Tamaño |T| = $\omega(G)$: 3

Subgrafo inducido por T ($|T| = 3$, $x = 114$)



RESOLUCIÓN — validación y respuesta corregida

Validado por IA

Los apartados a) y b) de arriba los resolvió el estudiante sin IA. Aquí la IA solo revisa esos resultados y, donde hace falta, los corrige.

Discusión — apartado a): la respuesta del estudiante es incorrecta (el método no es óptimo).

El procedimiento propuesto es válido como *construcción*: el conjunto que devuelve es realmente independiente (ninguna diferencia interna es un cuadrado). Pero es un algoritmo **voraz**, es decir, barre V de izquierda a derecha y nunca revisa una decisión ya tomada — y conservar un elemento pequeño puede obligar a descartar después varios elementos grandes. Un conjunto independiente *maximal* (al que no se le puede añadir nada) no tiene por qué ser *máximo* (el mayor de todos), y aquí no lo es: para $x = 114$ el voraz devuelve $|S| = 22$, mientras que el máximo real es $\alpha(G) = 27$.

La discrepancia no se ve en ejemplos pequeños: voraz y óptimo coinciden para todo $x \leq 36$, y la primera diferencia aparece en $x = 37$ (voraz 11, óptimo 12).

Respuesta corregida — a)

$$\alpha(G) = 27 \quad (x = 114),$$

con este certificado explícito de 27 elementos:

$S^* = [0, 2, 7, 10, 12, 15, 17, 20, 22, 34, 39, 41, 44, 52, 54, 65, 67, 72, 82, 87, 89, 94, 99, 104, 106, 109, 111]$

No hay fórmula cerrada sencilla para $\alpha(G)$ en función de x : con otro DNI hay que recalcularlo. Encontrar el máximo conjunto independiente es un problema NP-duro en general; aquí es abordable porque $|V| = 115$ es pequeño.

Discusión — apartado b): el resultado del estudiante es correcto; la justificación, incompleta.

$\omega(G) = 3$ es correcto, y se ha confirmado por búsqueda exhaustiva para *todos* los valores posibles de x ($50 \leq x \leq 149$, es decir, cualquier DNI). Dos matices sobre el razonamiento:

1. En el caso de 4 nodos se imponen 5 de las 6 diferencias: falta $|b - d|$. Para una demostración de *imposibilidad* esto no invalida nada (si ya es imposible pidiendo menos, también lo es pidiendo más), pero conviene decirlo explícitamente.
2. El punto débil real es dar por hecho que el caso más pequeño nace de la terna (3, 4, 5). Existen otras cadenas de ternas pitagóricas — (6, 8, 10), (8, 15, 17), (9, 12, 15)... — y otras formas de repartir cuál es la suma de cuáles; para cerrar el argumento habría que descartarlas todas, no solo la que arranca en (3, 4, 5). La comprobación exhaustiva del código cubre justamente ese hueco.

Respuesta b) — se mantiene: $\omega(G) = 3$, alcanzado por cualquier terna pitagórica dentro del rango, por ejemplo $T = \{0, 9, 25\}$.

CÓDIGO — verificación de la corrección

Validado por IA

```
In [5]: # =====
# VERIFICACIÓN — ¿es 22 realmente el máximo del apartado a)?
# =====
import itertools

# --- 1. Certificado: un conjunto independiente de 27 elementos ---
# Si este conjunto es válido, el 22 del voraz no puede ser el máximo.
S_optimo = [0, 2, 7, 10, 12, 15, 17, 20, 22, 34, 39, 41, 44, 52, 54, 65, 67, 72, 82, 87, 89, 94, 99, 104, 106, 109, 111]

es_independiente = all(abs(u - v) not in cuadrados for u, v in itertools.combinations(S_optimo, 2))

print(f"|S| voraz (apartado a) : {len(S)}")
print(f"|S*| certificado : {len(S_optimo)}")
print(f"¿S* es independiente? : {es_independiente}")
```

```

print(f">  $\alpha(G) \geq \{\text{len}(S_{\text{optimo}})\}$ , luego el voraz NO era óptimo")
print("-" * 60)

# --- 2. Voraz vs. óptimo exacto en casos pequeños ---
# El máximo exacto se obtiene como clique máximo del grafo COMPLEMENTARIO:
# un conjunto sin aristas en G es un conjunto con todas las aristas en el compl
def voraz(V, cuads):
    """Reproduce el algoritmo del apartado a) para un universo cualquiera."""
    Sv = V.copy()
    for k in range(len(Sv)):
        for i in [i for i in Sv if abs(i - Sv[k]) in cuads]:
            Sv.remove(i)
        if Sv.index(Sv[k]) >= len(Sv) - 1:
            break
    return Sv

def grafo_de(xx):
    """Construye el grafo del enunciado para un valor de x cualquiera."""
    V = list(range(xx + 1))
    cuads = {k**2 for k in range(1, int(xx**0.5) + 1)}
    Gx = nx.Graph()
    Gx.add_nodes_from(V)
    Gx.add_edges_from((i, j) for i in V for j in V if i < j and abs(i - j) in cuads)
    return Gx, V, cuads

print(" x    voraz    óptimo")
for xx in range(33, 41):
    Gx, V, cuads = grafo_de(xx)
    n_voraz = len(voraz(V, cuads))
    n_opt = len(nx.max_weight_clique(nx.complement(Gx), weight=None)[0])
    marca = " <-- se separan" if n_voraz != n_opt else ""
    print(f"{xx:>2}    {n_voraz:>5}    {n_opt:>6}{marca}")
print("-" * 60)

# --- 3.  $\omega(G) = 3$  para TODOS los DNI posibles (x de 50 a 149) ---
omegas = {max(len(c) for c in nx.find_cliques(grafo_de(xx)[0])) for xx in range(50,
print(f"Valores de  $\omega(G)$  observados para x en [50, 149]: {omegas}")

# --- 4. Cálculo exacto de  $\alpha(G)$  para nuestro x (lento: ~5 min) ---
# Desactivado por defecto; ponlo a True si quieres reproducir el 27 desde cero.
EJECUTAR_EXACTO = False
if EJECUTAR_EXACTO:
    S_exacto, _ = nx.max_weight_clique(nx.complement(G), weight=None)
    print(f" $\alpha(G)$  exacto =  $\{\text{len}(S_{\text{exacto}})\}$  -> {sorted(S_exacto)}")

```

```
|S| voraz (apartado a) : 22
|S*| certificado      : 27
¿S* es independiente? : True
=> α(G) >= 27, luego el voraz NO era óptimo
```

```
-----
x   voraz   óptimo
33    10     10
34    11     11
35    11     11
36    11     11
37    11     12 <-- se separan
38    11     12 <-- se separan
39    12     12
40    12     12
-----
```

Valores de $\omega(G)$ observados para x en $[50, 149]$: {3}

AMPLIACIÓN (explorada con IA)

1. ¿Hasta dónde aguanta $\omega(G) = 3$? El enunciado solo necesita el rango $x \leq 149$, pero cabe preguntarse si con un universo mucho mayor aparecería algún T de 4 elementos. Buscar cuádruplas $\{a, b, c, d\}$ con las **seis** diferencias cuadradas es equivalente a encadenar ternas pitagóricas cumpliendo todas las condiciones a la vez, y la búsqueda del código no encuentra ninguna hasta 50 000. Es decir: $\omega(G) = 3$ no es una peculiaridad del rango del DNI, sino algo mucho más robusto. (El problema general — ¿existen cuatro enteros con las seis diferencias cuadradas? — es una cuestión difícil de teoría de números, no un simple ejercicio de búsqueda.)

2. ¿Qué fracción del universo cabe en S ? Comparando el voraz con el óptimo exacto para x pequeño se ve dónde empieza a fallar la estrategia voraz y hacia qué *densidad* $\alpha(G)/(x+1)$ tiende el máximo: en torno a un 30–35 % del universo, no un valor que crezca sin control.

Ambas exploraciones las propuso y ejecutó la IA a petición del estudiante; los resultados son reproducibles con la celda siguiente.

```
In [6]: # =====
#  AMPLIACIÓN 1 – ¿existe algún T de 4 elementos en un rango grande?
#  =====
#  Truco: si el conjunto es {a, b, c, d}, podemos trasladarlo para que a = 0.
#  Entonces b, c y d son cuadrados, y además lo son sus diferencias mutuas.
N = 50_000
cuad_N = {k**2 for k in range(int(N**0.5) + 2)}
lista_cuad = sorted(s for s in cuad_N if 0 < s <= N)

cuadruplas = [(0, b, c, d)
               for i, b in enumerate(lista_cuad)
               for c in lista_cuad[i + 1:] if (c - b) in cuad_N]
```

```

        for d in lista_cuad if d > c and (d - b) in cuad_N and (d - c) in cua

print(f"Cuádruplas con las 6 diferencias cuadradas en [0, {N:}] : {len(cuadruplas)}")
print("=>  $\omega(G) = 3$  se mantiene mucho más allá del rango del enunciado")
print("-" * 60)

# =====
# AMPLIACIÓN 2 – voraz vs. óptimo, y densidad del máximo
# =====
xs = list(range(5, 41))
n_voraces, n_optimos = [], []
for xx in xs:
    Gx, V, cuads = grafo_de(xx)
    n_voraces.append(len(voraz(V, cuads)))
    n_optimos.append(len(nx.max_weight_clique(nx.complement(Gx), weight=None)[0]))

densidades = [a / (xx + 1) for a, xx in zip(n_optimos, xs)]

# Dos gráficas: tamaños comparados y densidad del máximo
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 3.6))

ax1.plot(xs, n_optimos, marker="o", ms=3, label="óptimo  $\alpha(G)$ ")
ax1.plot(xs, n_voraces, marker="s", ms=3, ls="--", label="voraz (apartado a)")
ax1.set_xlabel("x"); ax1.set_ylabel("tamaño de S"); ax1.legend(); ax1.grid(alpha=.3)
ax1.set_title("El voraz se queda corto a partir de x = 37")

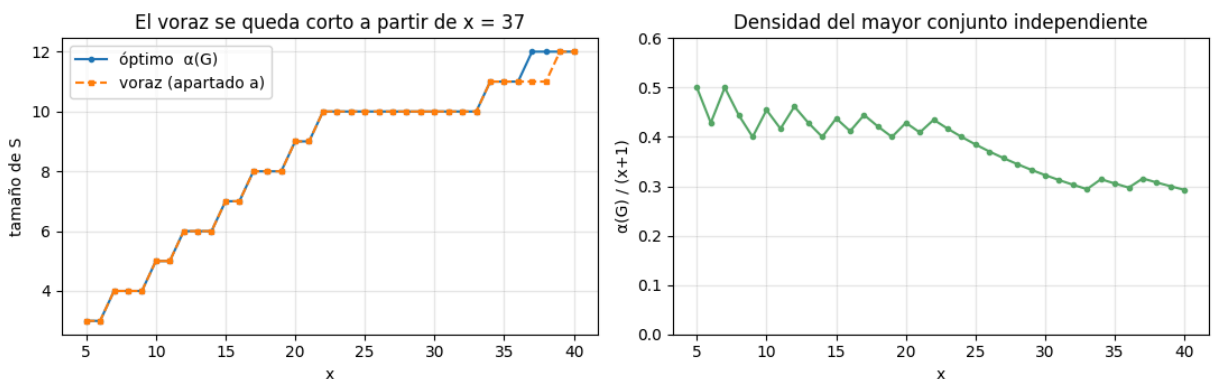
ax2.plot(xs, densidades, marker="o", ms=3, color="#55A868")
ax2.set_xlabel("x"); ax2.set_ylabel(" $\alpha(G) / (x+1)$ "); ax2.set_ylim(0, 0.6); ax2.grid()
ax2.set_title("Densidad del mayor conjunto independiente")

plt.tight_layout(); plt.show()

print(f"Densidad media para x en [20, 40]: {sum(densidades[-21:]) / 21:.3f}")

```

Cuádruplas con las 6 diferencias cuadradas en [0, 50,000] : 0
=> $\omega(G) = 3$ se mantiene mucho más allá del rango del enunciado



Densidad media para x en [20, 40]: 0.345



REGISTRO DE USO DE IA

- **Herramienta(s) utilizada(s):** Claude Code (modelo Claude Opus 5).
- **Prompt(s) utilizado(s):**
 1. "Prepara el fichero de la entrega 'Práctica 0' con la plantilla" + enunciado literal del boletín — la IA rellenó *Enunciado*, *Teoría* y *Datos*, y se detuvo.
 2. "Prepara los bloques vacíos para la resolución y el código y espera a que el estudiante lo intente por sí mismo".
 3. "El alumno ha rellenado el código y la resolución del apartado a). Solo quiero que lo arregles visualmente: la resolución en LaTeX y el código ordenado, comentado y con prints estéticos" (ídem después para el apartado b).
 4. "Rellena la Discusión sobre si la respuesta de la IA es correcta, la respuesta corregida y las ampliaciones del problema".
- **Fase del ejercicio en la que se usó IA:** (i) preparación del material previo (teoría y datos); (ii) *maquetación* — paso a LaTeX y limpieza del código, sin alterar el razonamiento; (iii) *verificación* de los resultados ya obtenidos por el estudiante; (iv) ampliaciones. **La resolución matemática de a) y b) es del estudiante, sin IA.**
- **Verificación de no-alucinación realizada:**
 - [x] Resultado verificado computacionalmente con una implementación independiente.
 - [x] Resultado contrastado analíticamente a mano.
 - [] Resultado contrastado con un caso/ejemplo conocido de la bibliografía.
 - Detalle: la corrección de a) no se acepta "porque lo diga la IA", sino porque viene con un **certificado comprobable** — un conjunto explícito de 27 elementos cuya independencia verifica el código en dos líneas; eso basta para probar que 22 no era el máximo. La optimalidad del 27 se obtiene por un método distinto al del estudiante (clique máximo exacto del grafo complementario, `nx.max_weight_clique`). Para b), $\omega(G) = 3$ se comprueba por búsqueda exhaustiva en todo el rango $x \in [50, 149]$, y a mano sobre la terna $T = \{0, 9, 25\}$.
- **Modificaciones realizadas sobre la propuesta de la IA:** pendiente de revisión final del estudiante.
- **Declaración de autoría:** El razonamiento matemático y la decisión final han sido revisados y comprendidos por el/la estudiante, quien asume la responsabilidad del contenido entregado.

💬 **¿Alguna duda sobre este ejercicio?** Pregunta lo que necesites, no hace falta esperar al final.